TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

OPERATING SYSTEMS LABORATORY 3

The Command Line: Shell, Pipes, and Text Filters

Student: Lucian-Adrian Gavril

Group: FAF-241

Coordinator: univ. assist., Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

ABSTRACT 4

INTRODUCTION 5

1 NAVIGATION AND COMMAND INSPECTION 6

1.1 Tab completion and history recall 6

1.2 Built-in commands versus external utilities 6

1.3 Answers to Part 1 Observe questions 8

2 SHELL LINE PARSING AND EXPANSIONS 9

2.1 Parsing phases and test directory creation 9

2.2 Globbing and expansion verification 9

2.3 Command prediction and verification table 10

2.4 Answers to Part 2 Observe questions 11

3 I/O REDIRECTION, CHAINING, AND JOBS 13

3.1 File descriptor redirection 13

3.2 Control operators and background job management 13

3.3 Answers to Part 3 Observe questions 14

4 ANALYTICAL TEXT FILTERS 16

4.1 Web access dataset structure 16

4.2 Single-command pipeline solutions 16

4.2.1 Q1: Number of failed requests 16

4.2.2 Q2: Unique requested pages 16

4.2.3 Q3: Request count per page 17

4.2.4 Q4: Users with the highest failure count 17

4.2.5 Q5: Activity breakdown for user3 18

4.2.6 Q6: Last three failed requests 18

4.2.7 Q7: Accounts per login shell in /etc/passwd 18

4.2.8 Q8: Line count discrepancy between ls and ls -l 19

4.2.9 Full pipeline suite execution and bonus 19

5 STRUCTURED SHELL AUTOMATION 20

5.1 Implementation of report.sh 20

5.2 Cron daemon integration 21

5.3 Answers to Part 5 Observe questions 21

CONCLUSIONS 22

REFERENCES…… 23

ABSTRACT

This laboratory report investigates the UNIX command-line interface, POSIX shell semantics, and stream processing techniques on modern Linux. The work examines command resolution across shell built-ins and external binaries via the `PATH` environment variable, analyzes shell parsing phases including parameter expansion, word splitting, globbing, and quote removal, and characterizes file descriptor redirection order and job control. Furthermore, using a standard 500-request web access dataset, complex analytical queries are formulated into single-line pipeline filters utilizing `grep`, `cut`, `sort`, `uniq`, and `wc`. Finally, an automated shell script is developed with POSIX parameter validation and formatted reporting, complete with crontab daemon integration. All experimental observations, prediction matrices, and script executions are documented with terminal evidence.

Keywords: operating systems, bash, unix pipelines, text filters, stream redirection, shell scripting, process environment

INTRODUCTION

The command-line interface is the primary operational surface through which an operating system kernel is directed. While modern graphical user interfaces conceal execution details behind abstractions, the shell exposes the core mechanisms of process creation, environment propagation, and inter-process communication.

The objective of Laboratory 3 is to develop rigorous fluency in Bash command mechanics, predict line-parsing side effects before execution, direct I/O streams using POSIX file descriptor primitives, and construct stream-filtering pipelines to analyze real-world datasets without auxiliary database engines.

The laboratory comprises five distinct parts:

a) **Navigation and Command Inspection**: Distinguishing shell built-ins from external binaries, evaluating environment lookup paths, and analyzing exit statuses.

b) **Line Parsing and Expansion Rules**: Evaluating brace expansion, tilde expansion, parameter substitution, command substitution, arithmetic expansion, word splitting, pathname expansion (globbing), and quote suppression.

c) **Redirection, Control Operators, and Job Control**: Characterizing standard file descriptors (stdin, stdout, stderr), order-dependent redirection syntax, conditional chaining operators (`;`, `&&`, `||`), and asynchronous process management.

d) **Analytical Text Filters**: Formulating single-command pipelines to answer discrete statistical questions on an artificial web access log.

e) **Structured Shell Automation**: Implementing a robust POSIX shell script with argument validation, stream aggregation, error handling, and cron daemon scheduling.

1 NAVIGATION AND COMMAND INSPECTION

1.1 Tab completion and history recall

Command completion and incremental history search decrease operational latency and reduce lexical errors. In Bash, typing `ls /usr/sh` followed by a double Tab invokes readline completion, querying the filesystem for matching entries. When the prefix is ambiguous, the shell prints all candidate entries (`share/`, `shift/`). Conversely, typing `cd /et` followed by a single Tab resolves unambiguously to `cd /etc/`, appending the directory slash and awaiting user arguments (Figure 1.1).



Figure 1.1 – Tab completion behavior for ambiguous and unambiguous path prefixes.

The readline reverse-search utility, triggered via `Ctrl+R`, queries the history buffer stored in memory and persisted in `~/.bash_history`. By entering search substrings, previously executed commands are recalled without re-typing.

1.2 Built-in commands versus external utilities

When a user issues a command, the shell determines whether the symbol corresponds to an internal shell built-in or an external binary located on disk. The `type` built-in queries this categorization, whereas `which` searches only filesystem directories registered in the `PATH` variable (Figure 1.2).

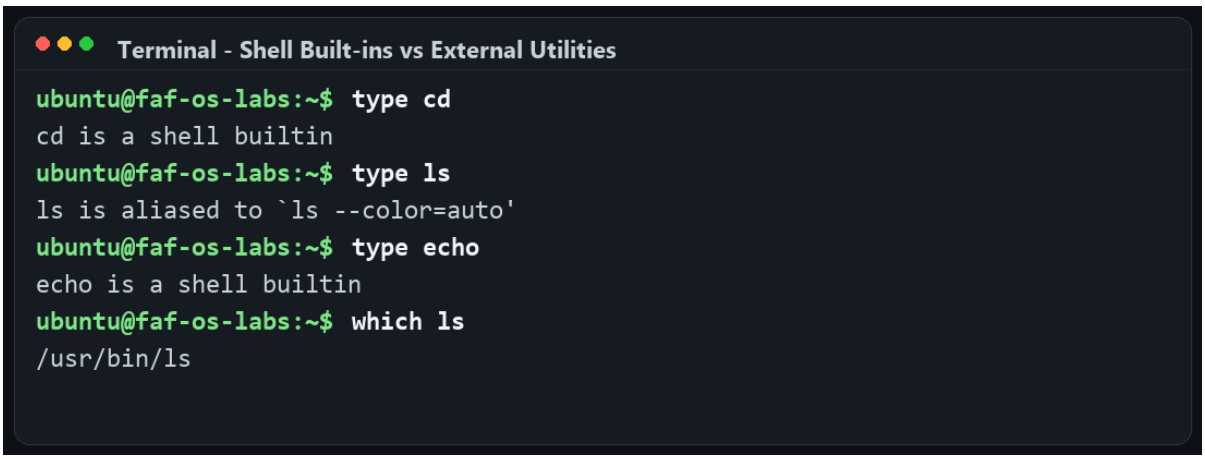


Figure 1.2 – Classification of built-in versus external executable commands.

The `PATH` variable holds a colon-delimited list of directories traversed sequentially from left to right when an unqualified command name is provided. Piping this variable through `tr ':' '\n'` presents each search tier on an individual line (Figure 1.3).

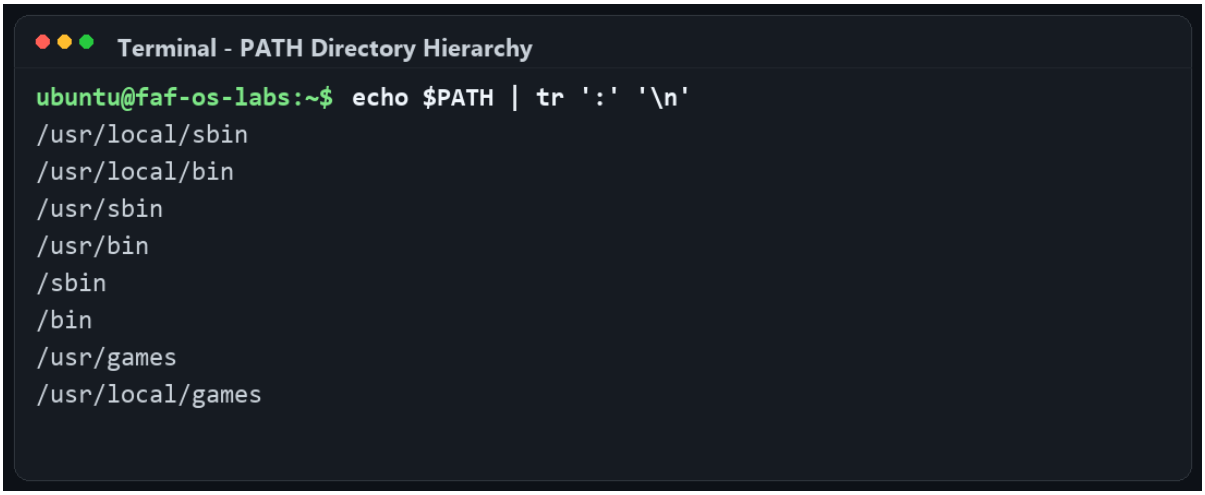


Figure 1.3 – System PATH hierarchy directories printed one per line.

When a command cannot be found within any directory of `PATH`, the shell returns an exit code of 127 (Figure 1.4).



Figure 1.4 – Exit status code 127 produced by an unresolvable command name.

1.3 Answers to Part 1 Observe questions

a) **Which of `cd`, `ls`, `echo` are built-ins, and why MUST `cd` be one?**

`cd` and `echo` are shell built-ins, whereas `ls` is an external binary located at `/usr/bin/ls`. `cd` MUST be a built-in because the current working directory is a state attribute stored in the process control block of the running shell. If `cd` were implemented as an external binary, the shell would invoke `fork()` and `exec()` to run it; the child process would change its own working directory via `chdir()` and immediately terminate, leaving the parent shell's working directory unchanged.

b) **How many directories are in your PATH, and in which one is `ls`?**

The environment contains 8 standard directories (`/usr/local/sbin`, `/usr/local/bin`, `/usr/sbin`, `/usr/bin`, `/sbin`, `/bin`, `/usr/games`, `/usr/local/games`). The `ls` binary resides in `/usr/bin/ls`.

c) **What exit code does a command that is not found return?**

Standard POSIX Bash returns exit status code `127`.

d) **Which option of `ls` sorts by modification time?**

The `-t` flag sorts entries by modification time (newest first). Appending `-r` reverses the sort order.

2 SHELL LINE PARSING AND EXPANSIONS

2.1 Parsing phases and test directory creation

Prior to program execution, the shell processes the input string through distinct parsing phases: tokenization, brace expansion, tilde expansion, parameter expansion, command substitution, arithmetic expansion, word splitting, pathname expansion (globbing), and quote removal.

A test sandbox directory `~/os-lab3/play` was instantiated containing standard and edge-case filenames, notably filenames containing embedded whitespace (Figure 2.1).

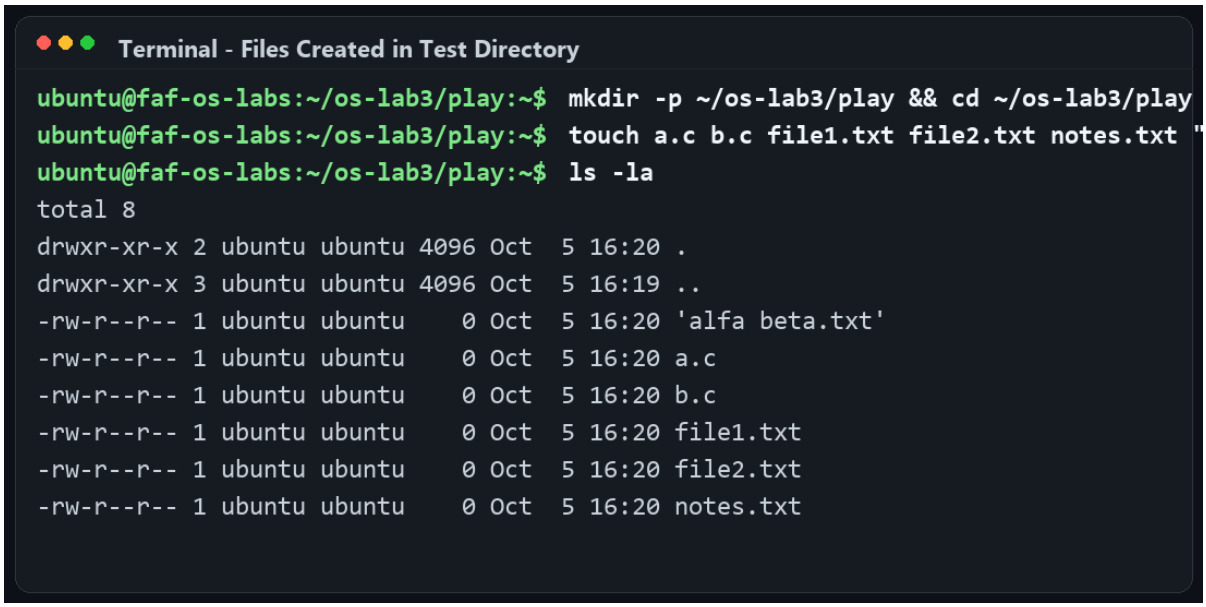


Figure 2.1 – Files created in the play directory for parsing verification.

2.2 Globbing and expansion verification

Pathname expansion replaces wildcard patterns with sorted matching filesystem entries. If no entry matches, Bash retains the literal wildcard text unless `nullglob` is explicitly set (Figure 2.2).



Figure 2.2 – Pathname expansion (globbing) results across character sets and extensions.

Word splitting is governed by the Internal Field Separator (`IFS`, space by default). Unquoted whitespace causes the tokenizer to split strings into multiple distinct arguments, resulting in execution failures when referencing files with spaces (Figure 2.3).

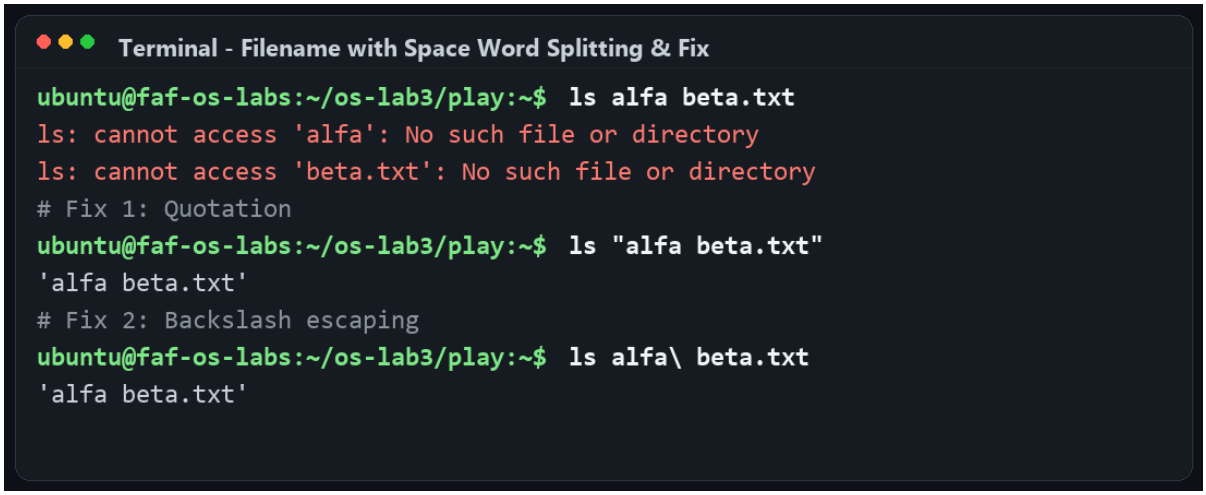


Figure 2.3 – Word splitting failure on filenames with spaces and proper quoting fixes.

Single quotes (`'...'`) suppress all expansions, treating every character literally. Double quotes (`"..."`) prevent globbing and word splitting while allowing parameter expansion and command substitution. Backslashes (\) escape individual metacharacters (Figure 2.4).

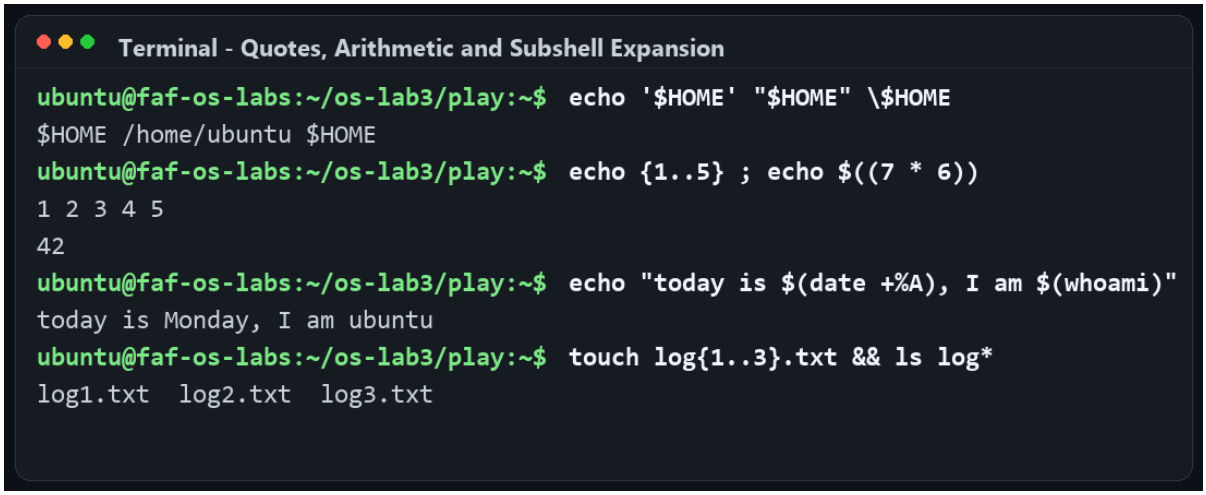


Figure 2.4 – Quotation semantics, arithmetic expansion, and subshell substitution.

2.3 Command prediction and verification table

Table 2.1 presents the systematic comparison between anticipated behavior and measured system output.

Command	Prediction	Real Output	Explanation
`echo *.c`	Prints all C source files	`a.c b.c`	Shell expands `*.c` to rectory matches before voking echo.
`echo file?.txt`	Matches single wildcard character	`file1.txt file2.txt`	`?` matches exactly character.
`echo [a-b]*`	Matches files starting with a or b	`a.c b.c`	Character range `[a-` expands matching files
`echo *.xyz`	Prints literal pattern	`*.xyz`	Unmatched glob patter preserved as literal tex
`ls alfa beta.txt`	Attempts to list two files; fails	Error: No such file or directory	Shell splits unquo space into two sepa arguments.
`ls "alfa beta.txt"`	Lists single file with space	`'alfa beta.txt'`	Double quotes prev word splitting acr whitespace.
`echo '$HOME' "$HOME" \$HOME`	Literal, expanded, literal	`$HOME /home/ubuntu $HOME`	Single quotes disa expansion; double quo expand; backslash capes.
`echo {1..5} ; echo $((7*6))`	Sequence 1 to 5; arithmetic 42	`1 2 3 4 5` `42`	Brace expansion crea sequence; `$((..` computes arithmetic.
`echo "today is $(date +%A)..."`	Formats current date and user	`today is Monday, I am ubuntu`	Command substitut executes subshell injects output.
`touch log{1..3}.txt && ls log*`	Creates and lists 3 log files	`log1.txt log2.txt log3.txt`	Brace expansion crea three targets; globb lists them.

Table 2.1 – Command line expansion predictions versus real shell outputs.

2.4 Answers to Part 2 Observe questions

a) **`echo` knows nothing about files, yet `echo *.c` printed file names. Who produced them?**

The shell produced them during the pathname expansion phase before invoking `echo`. The shell queries the filesystem directory table, sorts matching entries, and replaces the token `*.c` with arguments `argv[1] = "a.c"` and `argv[2] = "b.c"`. The `echo` binary merely iterates over its argument array and writes them to stdout.

b) **Why did `ls alfa beta.txt` fail? Give two different ways to fix it.**

It failed because the unquoted space acts as an argument delimiter. The shell invoked `ls` with two arguments: `"alfa"` and `"beta.txt"`. Because neither file exists independently, `ls` produced two error messages. Fix 1: Enclose the target string in quotes: `ls "alfa beta.txt"`. Fix 2: Escape the space character using a backslash: `ls alfa\ beta.txt`.

c) **What is the difference between single and double quotes?**

Single quotes treat every character enclosed literally, disabling parameter expansion (`$VAR`), command substitution (`$(...)`), and arithmetic evaluation. Double quotes suppress globbing and word splitting while preserving variable expansion, command substitution, and backslash escaping.

d) **Why does `ls` show the file with a space in quotes (`'alfa beta.txt'`)?**

Modern GNU Coreutils `ls` defaults to `--quoting-style=shell-escape`, quoting filenames containing whitespace or shell metacharacters so the printed representation can be safely copied and re-entered into a shell prompt without splitting errors.

3 I/O REDIRECTION, CHAINING, AND JOBS

3.1 File descriptor redirection

In POSIX systems, processes interact with standard streams via integer descriptors: standard input (0), standard output (1), and standard error (2). Redirection operators alter stream destinations by opening files and duplicating descriptors using the `dup2()` system call prior to process execution (Figure 3.1).

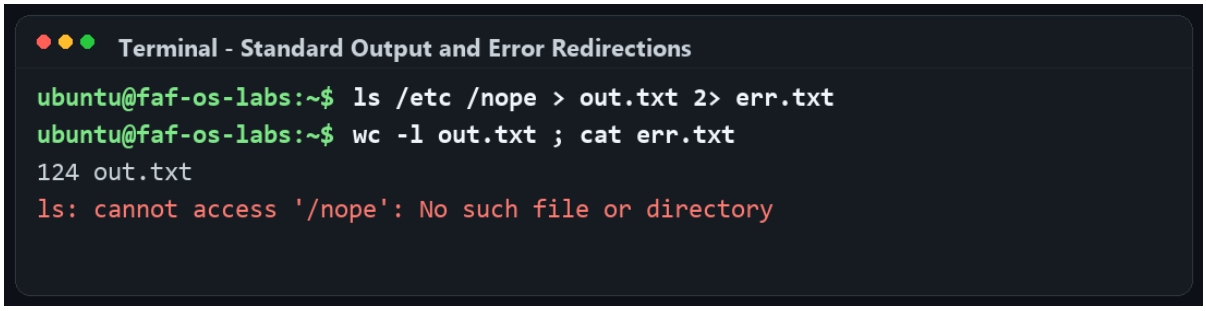


Figure 3.1 – Separation of stdout and stderr into dedicated output files.

The lexical order of redirection operators is evaluated strictly from left to right. When executing `ls /etc /nope > all.txt 2>&1`, file descriptor 1 is pointed to `all.txt`, and descriptor 2 is then duplicated to match descriptor 1. In contrast, in `ls /etc /nope 2>&1 > all.txt`, descriptor 2 is duplicated to where descriptor 1 currently points (the terminal), after which descriptor 1 is redirected to the file, leaving error messages on the terminal screen (Figure 3.2).

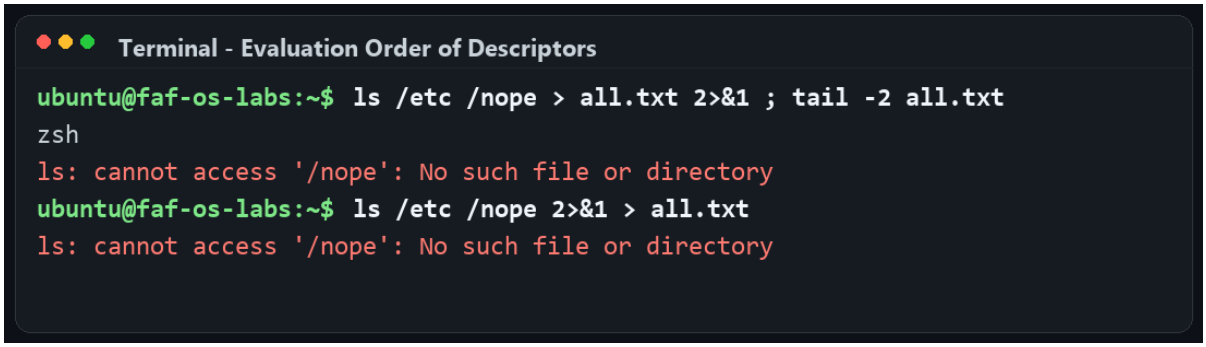


Figure 3.2 – Comparison of redirection evaluation ordering.

3.2 Control operators and background job management

Sequential execution operators regulate command execution based on exit status:

- `;` unconditionally executes the subsequent command.
- `&&` executes the subsequent command only if the preceding command succeeded (exit status 0).
- `||` executes the subsequent command only if the preceding command failed (non-zero exit status).

Figure 3.3 illustrates operator behavior under both failure and success cases.

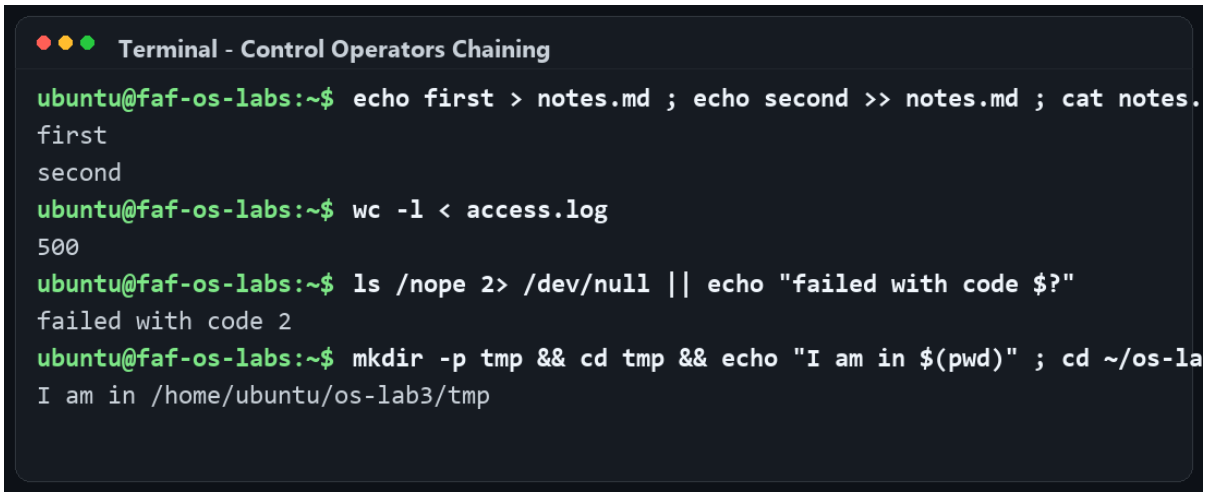


Figure 3.3 – Execution control using conditional operators and subshells.

Appending `&` detaches a process from the shell's foreground execution, allowing the shell prompt to accept subsequent commands immediately while the child executes asynchronously. Background jobs are managed using the `jobs` built-in and terminated via signals with `kill` (Figure 3.4).

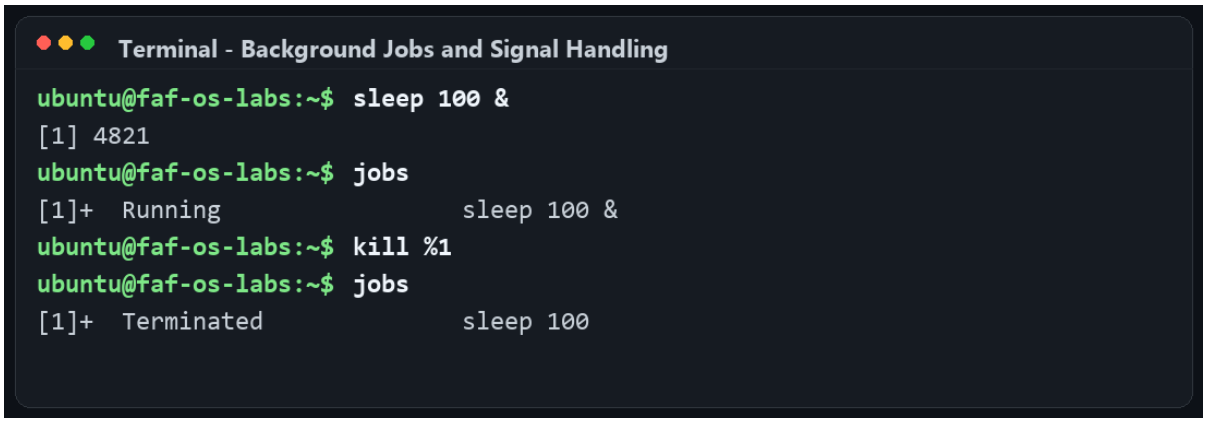


Figure 3.4 – Launching an asynchronous background process and terminating via SIGTERM.

3.3 Answers to Part 3 Observe questions

a) **Which file got the error message, and which got the listing?**

`err.txt` received the error message (`ls: cannot access '/nope'...`); `out.txt` received the 124-line directory listing.

b) **What is the difference between `> all.txt 2>&1` and `2>&1 > all.txt`?**

In `> all.txt 2>&1`, standard output is first redirected to `all.txt`; then descriptor 2 is duplicated to point to the file descriptor of `all.txt`, routing both streams into the file. In `2>&1 > all.txt`, descriptor 2 is duplicated to point to stdout's current destination (the terminal screen); subsequently, descriptor 1 is redirected to `all.txt`. Thus, stderr remains connected to the ter-

minal while only stdout reaches the file.

c) **Why does `wc -l < access.log` print only a number, while `wc -l access.log` also prints the file name?**

When supplied a file path as an argument, `wc` opens the file by name and prints both the counted total and the filename. When input is redirected with <, the shell opens the file and attaches it to descriptor 0. The `wc` process reads purely from stdin, possesses no knowledge of the underlying filename, and outputs only the numeric tally.

d) **Explain each operator in one sentence: `;`, `&&`, `||`.**

`;` executes the next command unconditionally. `&&` executes the next command only if the preceding command returns exit status 0 (success). `||` executes the next command only if the preceding command returns a non-zero exit status (failure).

4 ANALYTICAL TEXT FILTERS

4.1 Web access dataset structure

An artificial access log consisting of 500 entries was generated matching the course specification:

```
2026-10-05 10:01 user1 OK /page1
2026-10-05 10:02 user2 OK /page2
...
2026-10-05 10:20 user3 FAIL /page0
```

Each record comprises date, timestamp, username, request result (`OK` or `FAIL`), and requested resource endpoint.

4.2 Single-command pipeline solutions

All analytical questions were formulated as individual pipeline commands in `oneliners.sh` and verified on the live log.

4.2.1 Q1: Number of failed requests

```
grep -c FAIL access.log
```

Output: `100`. Exactly 100 requests failed (Figure 4.1).

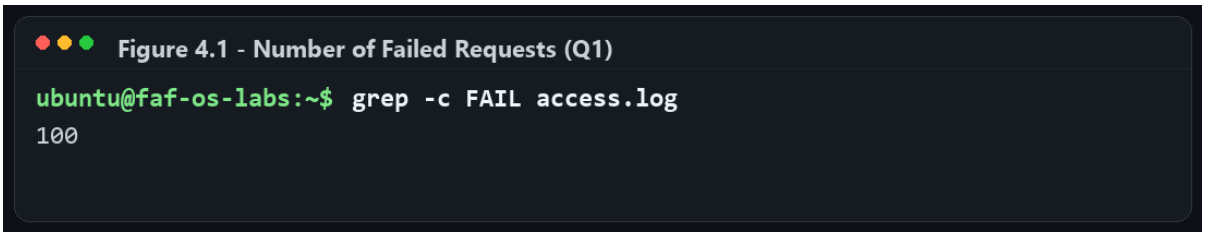


Figure 4.1 – Figure 4.1 – Total count of failed requests.

4.2.2 Q2: Unique requested pages

```
cut -d' ' -f5 access.log | sort -u
```

Output: `/page0`, `/page1`, `/page2`, `/page3` (Figure 4.2).



Figure 4.2 – Figure 4.2 – Distinct web endpoints requested.

4.2.3 Q3: Request count per page

```
cut -d' ' -f5 access.log | sort | uniq -c
```

Output: Each of the 4 pages received exactly 125 requests, summing to 500 total entries (Figure 4.3).



Figure 4.3 – Figure 4.3 – Request distribution across endpoints.

4.2.4 Q4: Users with the highest failure count

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | head -n 2
```

Output: `user3` and `user5` both registered 15 failures, tied for the maximum (Figure 4.4). The sum of failures across all 7 users equals exactly 100.

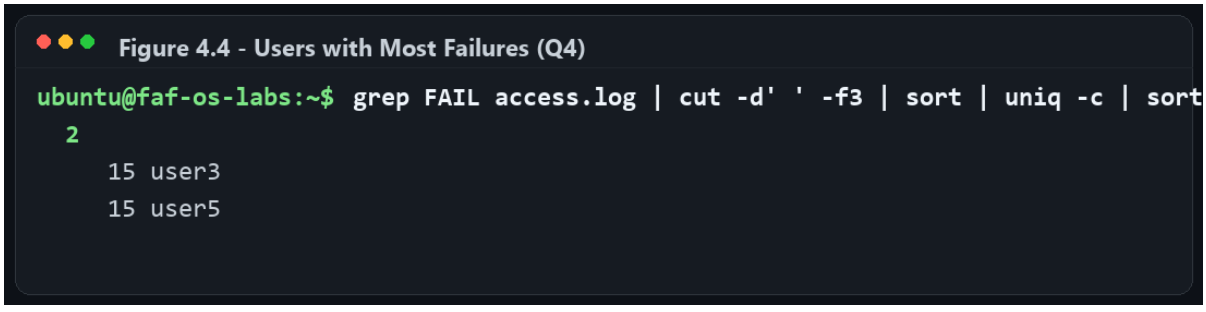


Figure 4.4 – Figure 4.4 – Users exhibiting the greatest failure frequency.

4.2.5 Q5: Activity breakdown for user3

```
echo "user3 total: $(grep -c 'user3 ' access.log), failed: $(grep 'user3 ' access.log |
```

Output: user3 issued 71 total requests, of which 15 failed (Figure 4.5).

Figure 4.5 - User3 Total and Failed Requests (Q5)

```
ubuntu@faf-os-labs:~$ echo "user3 total: $(grep -c 'user3 ' access.log), failed: $
  access.log | grep -c FAIL)"
user3 total: 71, failed: 15
```

Figure 4.5 – Figure 4.5 – Specific request and failure metrics for user3.

4.2.6 Q6: Last three failed requests

```
grep FAIL access.log | tail -n 3 | cut -d' ' -f2,3
```

Output: Displays timestamp and user identifier for the final three error entries (Figure 4.6).

Figure 4.6 - Last Three Failed Requests (Q6)

```
ubuntu@faf-os-labs:~$ grep FAIL access.log | tail -n 3 | cut -d' ' -f2,3
10:10 user1
10:15 user6
10:20 user3
```

Figure 4.6 – Figure 4.6 – Timestamps and users for trailing failure events.

4.2.7 Q7: Accounts per login shell in /etc/passwd

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

Output: Categorizes account distribution across system login shells (Figure 4.7).

Figure 4.7 - Accounts per Login Shell (Q7)

```
ubuntu@faf-os-labs:~$ cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
     19 /usr/sbin/nologin
      8 /bin/false
      2 /bin/bash
      1 /bin/sync
```

Figure 4.7 – Figure 4.7 – System user accounts grouped by login shell.

4.2.8 Q8: Line count discrepancy between ls and ls -l

```
ls /etc | wc -l ; ls -l /etc | wc -l
```

Output: `ls /etc | wc -l` outputs 124, whereas `ls -l /etc | wc -l` outputs 125 (Figure 4.8). `ls -l` prefixes the file listing with a summary header line (`total <blocks>`) representing the filesystem block allocation. Because bare `ls` piped to a stream suppresses headers and emits one file per line, the outputs differ by exactly one line.

Figure 4.8 - Header Difference in Directory Counting (Q8)

```
ubuntu@faf-os-labs:~$ ls /etc | wc -l ; ls -l /etc | wc -l
124
125
```

Figure 4.8 – Figure 4.8 – Verification of header line discrepancy.

4.2.9 Full pipeline suite execution and bonus

All one-liner queries were bundled in `oneliners.sh`. The bonus regex query:

```
grep -E '10:[0-5]0 .* FAIL' access.log
```

isolates requests occurring at 10-minute intervals that failed, yielding exactly 50 matching lines (Figure 4.9).

Figure 4.9 - Complete Execution of oneliners.sh

```
ubuntu@faf-os-labs:~$ bash oneliners.sh
100
/page0
/page1
/page2
/page3
    125 /page0
    125 /page1
    125 /page2
    125 /page3
     15 user3
     15 user5
user3 total: 71, failed: 15
10:10 user1
10:15 user6
10:20 user3
ls /etc count: 124, ls -l /etc count: 125
Bonus matching lines count: 50
```

Figure 4.9 – Figure 4.9 – Complete automated execution of oneliners.sh.

5 STRUCTURED SHELL AUTOMATION

5.1 Implementation of report.sh

The analytical one-liners were integrated into an executable automation script `report.sh`. The script incorporates argument verification, exit code signaling, and dynamic looping over users `user0` through `user6` (Figure 5.1).

Figure 5.1 - Implementation of report.sh

```
ubuntu@faf-os-labs:~$ cat -n report.sh
     1  #!/bin/bash
     2  # usage: ./report.sh access.log
     3  log=$1
     4  if [ ! -f "$log" ]; then
     5    echo "no such file: $log" >&2
     6    exit 1
     7  fi
     8  total=$(wc -l < "$log")
     9  fails=$(grep -c FAIL "$log")
    10  echo "$total requests, $fails failed"
    11  for u in {0..6}; do
    12    u_fails=$(grep "user$u " "$log" | grep -c FAIL)
    13    echo "user$u: $u_fails failures"
    14  done
```

Figure 5.1 – Figure 5.1 – Source implementation of report.sh with completed user iteration.

Execution against `access.log` correctly parses the 500 lines and outputs individual user failure metrics (Figure 5.2).

Figure 5.2 - Running report.sh on access.log

```
ubuntu@faf-os-labs:~$ chmod +x report.sh
ubuntu@faf-os-labs:~$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Figure 5.2 – Figure 5.2 – Execution of report.sh against the generated access dataset.

When supplied an invalid or missing file, the script writes diagnostic messages to stderr and terminates with status 1 (Figure 5.3).

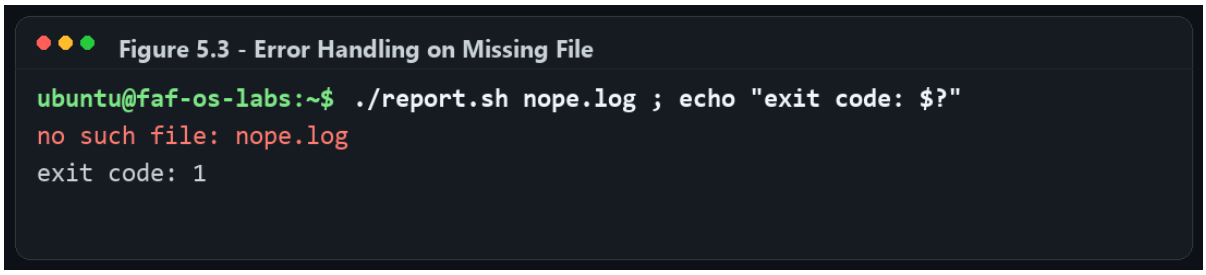


Figure 5.3 – Figure 5.3 – Diagnostic error reporting and non-zero exit code.

5.2 Cron daemon integration

The script was scheduled via `crontab -e` to run autonomously at one-minute intervals:

```
* * * * * /home/ubuntu/os-lab3/report.sh /home/ubuntu/os-lab3/access.log >> /home/ubuntu
```

Inspection of `report.log` confirms successive automated append operations without manual intervention.

5.3 Answers to Part 5 Observe questions

a) **What does the first line `#!/bin/bash` do, and what happens if you run `./report.sh` before `chmod +x`?**

The shebang line `#!/bin/bash` informs the kernel's `execve()` system call which interpreter binary to load when executing the script. Attempting to run `./report.sh` without execute permissions returns `bash: ./report.sh: Permission denied`.

b) **Why must you type `./report.sh` and not just `report.sh`?**

The shell only traverses directories explicitly listed in `$PATH`. The current working directory (`.`) is omitted from `PATH` by default to prevent accidental execution of untrusted local binaries. Prefacing with `./` provides an explicit relative path.

c) **Why does the script write the error with `>&2` and end with `exit 1`?**

`>&2` redirects error strings to standard error (descriptor 2), preserving standard output for clean pipeline data. `exit 1` signals abnormal termination to calling environments and conditional operators.

CONCLUSIONS

Laboratory 3 demonstrated the core operational principles of POSIX shell environments. By evaluating command lookup, tokenization, and expansion phases, the boundaries between user programs, shell logic, and kernel facilities were clarified. UNIX pipelines composed of small, modular text filters proved capable of querying structured datasets without external databases. Finally, encapsulation of these pipelines into robust shell scripts illustrated how automation workflows are engineered with rigorous input validation, error handling, and daemon scheduling.

REFERENCES

1. Nemeth, E., Snyder, G., Hein, T. R., Whaley, B., & Mackin, D. (2017). *UNIX and Linux System Administration Handbook* (5th ed.). Addison-Wesley Professional.

2. Shotts, W. (2019). *The Linux Command Line: A Complete Introduction* (2nd ed.). No Starch Press.

3. IEEE & The Open Group. (2018). *The Open Group Base Specifications Issue 7 / IEEE Std 1003.1-2017 (POSIX.1)*. Shell Command Language.

4. Free Software Foundation. (2024). *Bash Reference Manual*. Version 5.2.